SIMULASI KENDALI POSISI SUDUT SERVO *DYNAMIXEL* MENGGUNAKAN PID *CONTROLLER*

1.1 Pendahuluan

Motor servo merupakan aktuator yang digunakan untuk mengatur posisi sudut sesuai dengan nilai referensi yang diberikan. Motor servo banyak digunakan pada sistem robotika, seperti lengan robot, robot *humanoid*, robot *hexapod* dll. Pada proyek ini dilakukan simulasi kendali posisi sudut motor servo menggunakan metode *Proportional*, *Integral*, dan *Derivative* (PID) dengan *Python* 3. Sudut referensi ditentukan sebesar 90˚, sedangkan posisi awal motor adalah 0˚. Pengendali PID menghitung error berdasarkan selisih antara sudut referensi dan posisi aktual, kemudian menghasilkan sinyal kendali untuk menggerakkan motor menuju sudut target sudut.

Simulasi menggunakan sistem umpan balik (*feedback*), yaitu posisi aktual motor diperiksa secara berkala dan dibandingkan dengan referensi. Dengan hal itu, respon sistem dapat diamati berdasarkan kemampuan mencapai target sudut, *rise time*, *overshoot*, *peak time*, *settling time*, *steady state error*.

1.2 Penjelasan Sistem Kendali

Sistem kendali yang digunakan merupakan sistem kendali umpan balik kalang tertutup (*closed-loop control*). Posisi sudut aktual dibandingkan dengan sudut referensi untuk menghasilkan *error*. Pengendali PID mengolah *error* tersebut dan menghasilkan sinyal kendali yang digunakan untuk menggerakkan motor servo.

1.3 Persamaan Kendali PID

$$u(t) = K_p e(t) + K_i \int_0^t e\,(\tau) d\tau + K_d \frac{de(t)}{dt}$$

1.4 Model matematis motor servo

$$\tau \frac{d\theta(t)}{dt} = Ku(t) - \theta(t)$$

1.5 Flowchart/Diagram Alir



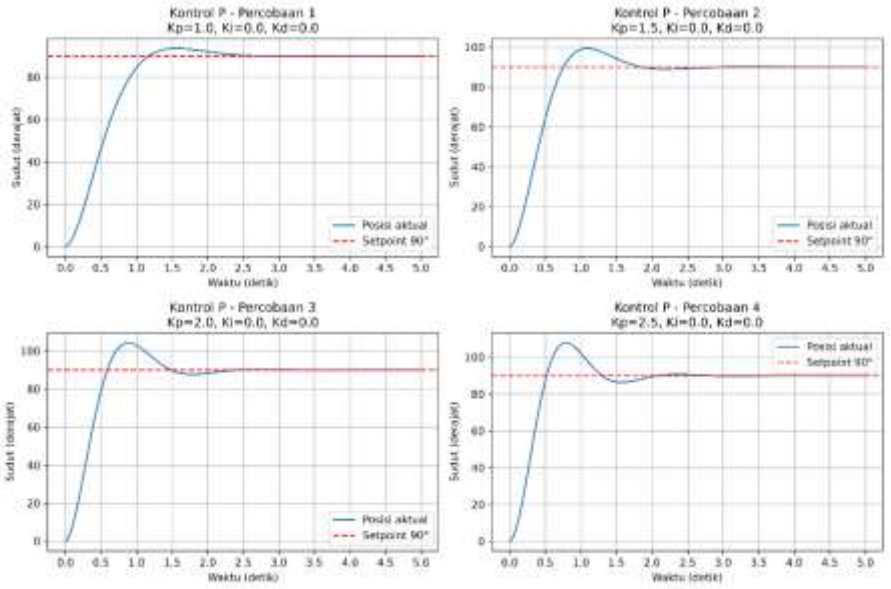
1.6 Alat dan Bahan

Hardware	Laptop
Software	*Visual Studio Code*

1.7 Hasil dan Pembahasan

1.7.1 Pengujian Sistem Dengan Kendali Proporsional (P)

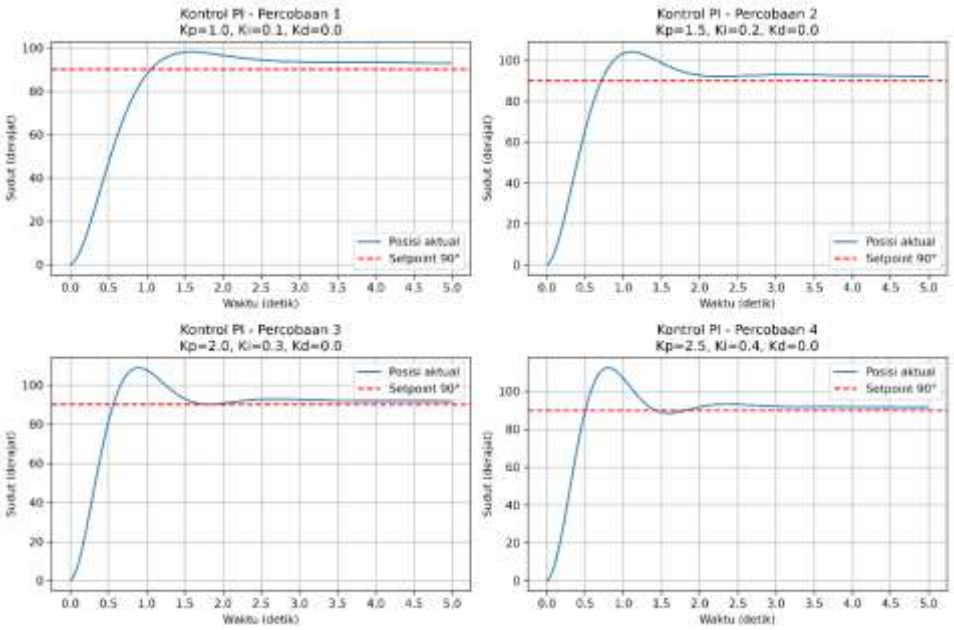
Uji ke	**Kp**	**Ki**	**Kd**	***Rise Time* (TR)**	***Overshoot* (Mp)**	***Peak Time* (TS)**	***Settling Time* (ST)**	***Steady State Errror***
1	1,0	0,0	0,0	0,930	4,05	1,560	2,070	0,005
2	1,5	0,0	0,0	0,650	10,48	1,090	1,680	0,002
3	2,0	0,0	0,0	0,520	15,91	0,890	1,980	0,001
4	2,5	0,0	0,0	0,470	19,82	0,790	1,860	0,003



Dari hasil pengujian diatas, meningkatnya nilai K_p dari 1,0 menjadi 2,5 menyebabkan respon sistem semakin cepat. Hal ini ditunjukkan oleh penurunan *rise time* dari 0,930 detik menjadi 0,470 detik dan *peak time* dari 1,560 detik menjadi 0,790 detik. Namun, peningkatan nilai K_p juga menyebabkan *overshoot* meningkat dari 4,05% menjadi 19,82%, yang berarti sudut servo semakin besar melewati nilai *setpoint* sebelum stabil. Nilai *settling time* mengalami perubahan yang tidak konsisten, sedangkan *steady-state error* tetap sangat kecil, antara 0,001 hingga 0,005 derajat. Dengan data tersebut, semakin besar nilai K_p, maka semakin cepat respon sistem, namun risiko terjadinya *overshoot* juga semakin tinggi. Jadi, pemilihan nilai K_p perlu mempertimbangkan keseimbangan antara kecepatan respon dan kestabilan sistem.

1.7.2 Pengujian Sistem Dengan Kendali Proporsional Integral (PI)

Uji ke	**Kp**	**Ki**	**Kd**	***Rise Time* (TR)**	***Overshoot* (Mp)**	***Peak Time* (TS)**	***Settling Time* (ST)**	***Steady State Errror***
1	1,0	0,1	0,0	0,880	9,10	1,580	NaN	3,037
2	1,5	0,2	0,0	0,630	15,67	1,100	NaN	2,200
3	2,0	0,3	0,0	0,500	20,88	0,890	4,550	1,670
4	2,5	0,4	0,0	0,460	24,87	0,800	4,050	1,502

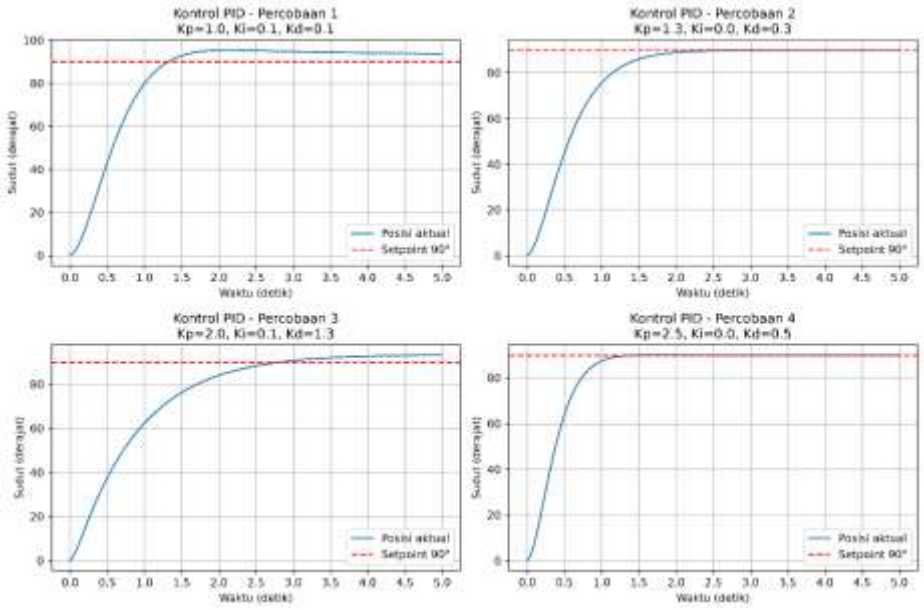


Dari hasil empat percobaan kontrol *integral* (I), peningkatan nilai K_p dari 1,0 menjadi 2,5 dan K_i dari 0,1 menjadi 0,4 mempunyai efek respon sistem semakin cepat, ditunjukkan oleh penurunan *rise time* dari 0,880 detik menjadi 0,460 detik dan *peak time* dari 1,580 detik menjadi 0,800 detik. Selain itu, *steady-state error* menurun dari 3,037 derajat menjadi 1,502 derajat, menunjukkan bahwa keluaran sistem semakin mendekati nilai target. Namun, *overshoot* meningkat dari 9,10% menjadi 24,87%, sehingga respon sistem cenderung melewati target lebih jauh. Percobaan 4 menghasilkan respon tercepat dan kesalahan keadaan tunak terkecil, sedangkan percobaan 1 memiliki *overshoot* paling rendah. Nilai *settling time* yang tercatat *NaN* pada percobaan 1 dan 2 menunjukkan bahwa waktu stabil belum terdeteksi oleh program berdasarkan kriteria yang digunakan. Secara keseluruhan, peningkatan parameter menyebabkan respon lebih cepat dan mengurangi kesalahan, tetapi perlu diimbangi dengan pengaturan yang tepat agar *overshoot* tidak terlalu besar dan sistem dapat mencapai kondisi stabil.

1.7.3 Pengujian Sistem Dengan Kendali Proporsional Integral Derivativ (PID)

Uji ke	**Kp**	**Ki**	**Kd**	***Rise Time* (TR)**	***Overshoot* (Mp)**	***Peak Time* (TS)**	***Settling Time* (ST)**	***Steady State Errror***
1	1,0	0,1	0,1	1,030	6,09	2,130	NaN	3,767

Uji ke	**Kp**	**Ki**	**Kd**	***Rise Time* (TR)**	***Overshoot* (Mp)**	***Peak Time* (TS)**	***Settling Time* (ST)**	***Steady State Errror***
2	1,3	0,0	0,3	1,190	0,00	4,990	1,800	0,000
3	2,0	0,1	0,0	0,500	20,88	0,890	4,550	1,670
4	2,5	0,0	0,5	0,760	0,09	1,680	1,060	0,000



Berdasarkan hasil simulasi, penggunaan nilai integral $(K_i = 0)$ pada percobaan 4 menghasilkan respon sistem yang paling baik dibandingkan percobaan lainnya. Hal ini ditunjukkan oleh *overshoot* yang sangat kecil, yaitu 0,09%, *settling time* sebesar 1,060 detik, dan *steady-state error* sebesar 0 derajat. Nilai $(K_i = 0)$ berarti komponen *integral* tidak aktif, sehingga sistem hanya menggunakan kendali *proportional* (P) dan *derivative* (D). Pada percobaan ini, kombinasi $(K_p = 2{,}5)$ dan $(K_d = 0{,}5)$ menghasilkan respon yang paling baik yaitu cepat, stabil, dan mendekati target tanpa adanya komponen *integral*. Jadi, penggunaan nilai *integral* nol(0) dapat diterima karena berdasar pada hasil simulasi yang diperoleh. Tetapi, tidak berarti komponen *integral* selalu tidak diperlukan, karena pada sistem lain *integral* dapat membantu mengurangi kesalahan yang masih tersisa.

1.8 Link Source Code

https://github.com/MuhammadZandaru/Tugas-Project-UTS-MSK-2026